

Algoritmos e Lógica da Programação - Lucy Mari

Nome do projeto: APRENDER+

Integrantes: Bruno Sales, Gabriel Barbosa, Mel Oliva e Rayna Froes

Professor(a) orientador(a): Eduardo Savino

Matrizes

Uma matriz é um conjunto de variáveis de mesmo tipo, com mesmo nome e alocadas sequencialmente na memória. Por terem o mesmo nome, o que as distingue são índices que mostram sua localização dentro da estrutura. Elas podem ser tanto unidimensionais (vetores) como multidimensionais.

Declarar matriz:

<tipo da matriz> [<linha>] [<coluna>];

Exemplo:

inteiro [3] [4];

Exemplo matriz apresentada no código do PI:

```
for (int i = 0; i < acessos.GetLength(0); i++)
{
    if (acessos[i, 0] == ipEntrada)
    {
        Console.WriteLine("Status encontrado na matriz: " + acessos[i, 1]);
    }
}
```

Entrada: IP digitado pelo usuário

Processamento: comparação do IP com a matriz

Saída: status encontrado na matriz: AUTORIZADO (se o ip for encontrado)

Vetores

Um vetor é um conjunto de informações de um mesmo tipo de dado. Se possuímos um vetor chamado **Vet**, com cinco posições: 0, 1, 2, 3 e 4. Na posição 0 do vetor Vet, temos a variavel1, na posição 1 do vetor Vet, temos a variavel2, até que na posição 4 do vetor Vet, temos a variavel5.

Declarar vetor:

<nome do vetor> [<numero de elementos>] <tipos de elementos do vetor>;

Exemplo:

nome_vet [100] inteiro;

Atribuição de valores:

<nome do vetor> [<posição>] ← <valor>;

Exemplo:

nome_vet [0] ← 1;

É possível também **atribuir valores no momento da declaração** do vetor, conforme a seguir:

<nome do vetor> [<número de dados>] ← {<valor1>,...<valorn>} <tipos de dados do vetor>;

Exemplo:

nome_vet [3] ← {1, 2, 3} inteiro;

Para **mostrar os elementos do vetor** é escrito:

escrever (<nome do vetor>[<posição>]);

Exemplo:

escrever(nome_vet[0]);

escrever("O valor é ", nome_vet[0])

Exemplo vetores apresentados no código PI:

```
// Lista de IPs autorizados (simulação)
List<string> ipsAutorizados = new List<string>()
{
    "192.168.0.1",
    "10.0.0.5"
};
```

Entrada: valores guardados na lista de IPs autorizados

Processamento: verifica se o IP digitado existe na lista

Saída: autorização ou bloqueio de acesso

Modularização

A divisão do algoritmo em módulos ou subrotinas, para que o problema dividido em subproblemas possa ser facilmente interpretado e desenvolvido. Temos também o método de refinamento sucessivo que é quando o problema complexo é dividido em problemas menores e, a partir do resultado dos problemas menores, temos o resultado do problema complexo.

Uma variável pode ter o escopo dividido em dois tipos: variável global ou variável local.

Variável global é aquela declarada no início do algoritmo e pode ser utilizada por qualquer parte deste algoritmo, seja nos comandos ou dentro de qualquer módulo que o pertença. Sua declaração é feita apenas uma única vez e não é permitido que o mesmo nome de variável seja declarado dentro de qualquer outra parte do algoritmo, por exemplo, dentro de qualquer outro módulo.

Variável local é aquela declarada dentro de algum bloco, por exemplo, dentro de um módulo. Neste caso, esta variável é válida e reconhecida somente dentro do bloco em que foi declarada. o mesmo nome de variável pode ser declarado dentro de diferentes blocos, pois serão reconhecidas como uma nova variável.

Módulo Procedimento

São os módulos que não retornam valor para o módulo ou algoritmo que os tenham chamado. Ele é utilizado quando o algoritmo principal ou módulo que chama o módulo procedimento não necessita de retorno de qualquer dado do módulo.

Ex. mód. procedimento:

início_algoritmo

<nome do módulo> ()

início

<comandos>;

fim módulo;

Ex. aplicado:

Dados ()

início

Declarar

disc, fac alfanumérico;

disc $\leftarrow$ "Lógica de Programação";

```
fac ← "FECAP";
```

```
escreva ("nome da disciplina: " , disc);
```

```
escreva ("nome da instituição de ensino: " , fac);
```

```
fimmódulo;
```

Chamada do módulo no algoritmo principal:

```
<nome do módulo> ();
```

Dentro dos parênteses os **argumentos** são passados como **parâmetros** aos módulos. Cada argumento deverá ser declarado no momento que o módulo for desenvolvido.

Exemplo com parametrização:

```
<nome do módulo> (<tipo da var> <nome da var> , ... , <tipo da var> <nome da var>)
```

```
início
```

```
<comandos>;
```

```
fimmódulo;
```

Exemplo aplicado (com parametrização):

```
Dados (alfanumérico disc , alfanumérico fac)
```

```
início
```

```
escreva ("nome da disciplina => " , disc);
```

```
escreva ("nome da instituição de ensino =>" , fac);
```

```
fimmódulo;
```

Chamada do módulo com parametrização:

```
<nome do módulo> (<valor ou variável > , ... , <valor ou variável >);
```

Modulo função

São os módulos que retornam algum valor para o módulo ou algoritmo que os tenham chamado. Ele é utilizado quando o algoritmo principal ou módulo que chama o módulo função necessita de retorno de qualquer dado do módulo.

Ex. mód. função:

```
<tipo de retorno> <nome do módulo> ( )
```

início

<comandos>;

retornar <valor de retorno>;

fim módulo;

A palavra **retornar** é a principal palavra do módulo função, pois é a partir deste comando que um valor é retornado para o algoritmo ou módulo que o chamou. O valor de retorno enviado por esse comando precisa ser do mesmo tipo de dado declarado para o módulo função.

Exemplo aplicado:

alfanumérico Dados ()

início

Declarar

disc, fac, mens alfanumérico;

disc ← "Lógica de Programação";

fac ← "FECAP";

mens ← "nome da disciplina => " + disc + "\n nome da instituição de ensino =>" + fac;

retornar mens;

fim módulo;

Chamada do módulo no algoritmo principal:

<variável> ← <nome do módulo> ();

Ex. de modularização no código do PI:

```

private void CriarTabelas()
{
    if (conexao == null)
    {
        return;
    }
    string sqlJogos = @"
        CREATE TABLE IF NOT EXISTS JOGOS (
            codigo INTEGER NOT NULL PRIMARY KEY,
            nome TEXT NOT NULL,
            descricao TEXT NOT NULL,
            tema TEXT NOT NULL,
            status TEXT NOT NULL
        );";

    using (SQLiteCommand cmd = conexao.CreateCommand())
    {
        cmd.CommandText = sqlJogos;
        cmd.ExecuteNonQuery();
    }
}

```

Entrada: conexão com o banco de dados e comando SQL

Processamento: Verifica se a conexão está aberta e cria a tabela no banco SQLite

Saída: tabela criada ou que já existe no banco